

AI Behavior Tree Toolkit

Visual editor for authoring behavior trees and utility AI, with live in-engine debugging, blackboard inspection, and an Unreal runtime.

Unreal	C++	Blueprint	ImGui
--------	-----	-----------	-------

VERSION	STATUS	LICENSE	ENGINE	AGENTS	UPDATED
2.1.0	Open Source	MIT	Unreal 5.5–5.7	500+	Sep 2026

1. Overview

A toolkit for authoring agent behaviour in Unreal that combines classical behavior trees with utility scoring. Structure comes from the tree — sequences, selectors, decorators — while the choice between viable branches can be delegated to weighted considerations rather than fixed priority order. Designers get readable control flow without hand-ordering every fallback.

The debugging story is the reason to use it over the engine's built-in system. An ImGui overlay runs in-game showing the live tree with the active path highlighted, every blackboard key and its current value, and the utility scores that produced the last decision. Sessions can be recorded and scrubbed, so an agent that did something strange thirty seconds ago is inspectable after the fact instead of only reproducible.

2. Runtime tick

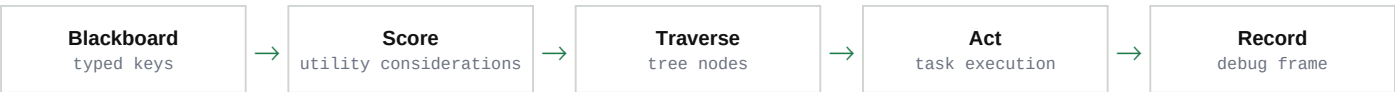


Figure 1 — Recording is always on in development builds and compiled out entirely in shipping, leaving no runtime cost.

Trees are ticked on a budget rather than every frame per agent. Agents are bucketed by distance and importance, with nearby agents ticking at full rate and distant ones at a fraction; a blackboard observer can promote an agent to full rate immediately when something relevant changes, so budgeting never delays a reaction that matters.

3. Components

Editor	Graph editor with node palette, live validation, and inline consideration curve editing.
Asset	Trees serialise as Unreal assets; diff and merge friendly text export for version control.
Runtime	Instanced node state per agent, pooled allocations, budgeted ticking by distance bucket.
Utility scorer	Named considerations with response curves, combined by weighted product with veto.
Blackboard	Typed keys, per-agent and shared scopes, observer callbacks for reactive decorators.
Debugger	In-game ImGui overlay, session recording, and scrubbable playback of past decisions.

4. Custom nodes

```
UCLASS()
class UBTTTask_TakeCover : public UBTTToolkitTask {
    GENERATED_BODY()
public:
    UPROPERTY(EditAnywhere, Category = "Cover")
    FBlackboardKeySelector ThreatKey;          // typed; editor validates the binding

    // Utility score, 0..1. Called only when the parent is a utility selector.
    float Score(const FBTContext& Ctx) const override {
        const float Exposure = Ctx.Get<float>("Exposure");
        const float Health   = Ctx.Get<float>("HealthPct");
        return Curve("exposure", Exposure) * Curve("low_health", Health);
    }

    EBTStatus Tick(FBTContext& Ctx, float Dt) override {
        if (!Ctx.Has(ThreatKey)) return EBTStatus::Failed;
        return MoveToCover(Ctx) ? EBTStatus::Running : EBTStatus::Failed;
    }
};
```

Listing 1 — Tasks written in C++ are exposed to Blueprint automatically, including their considerations.

5. Debugging workflow

- **Live overlay.** Selecting an agent in-game draws its tree with the active path, the status of every visited node, and the elapsed time in the current task.
- **Score inspection.** Utility selectors show each child's considerations, their curve outputs, and the final product, which turns "why did it choose that" into a readable table.
- **Recording.** The last N seconds of decisions, blackboard values, and scores are buffered and can be scrubbed or exported, so intermittent behaviour is diagnosable without a repro.

6. Performance

SCENARIO	COST	NOTES
500 agents, budgeted	1.8 ms/frame	Mixed buckets; 60 agents at full rate.
500 agents, unbudgeted	6.4 ms/frame	Every agent ticking every frame, for comparison.
Single tree tick	11 µs	24-node tree with two utility selectors.
Debug recording	0.3 ms/frame	Development builds only; absent in shipping.

7. Integration notes

- Installs as an engine plugin. It does not replace or depend on Unreal's built-in behavior tree system, and the two can run side by side during a migration.
- Blueprint-only projects can author trees and tasks entirely without C++, at roughly a three times per-node cost that is irrelevant at typical agent counts but shows up above a thousand agents.
- Tree assets export to a text format for review, so a behaviour change arrives in a pull request as a readable diff rather than as an opaque binary asset.

8. Roadmap

RELEASE	PLANNED WORK	TARGET
2.2	Shared squad blackboards with authority rules for coordinated behaviour.	Q4 2026
2.3	Headless simulation harness for regression-testing behaviour in CI.	Q1 2027
3.0	Hierarchical task network planner as an alternative node category.	Q2 2027